Practical -6

Aim:

Write a Program to implement error detection and correction using Hamming code concept make a-tat Run to input data Stream and verify error correction feature.

Error correction at Data link Layer.

Hamming code is a set of error correction codes that can be used to detect and correct the errors that can when the data is transmitted the sender to receives.

Create Sender Program:

1. Input to Sender bit should be a text of any length, Program should convert text to binary.
2. Apply Hamming code on the binary data and add redundant bits to it
3. Save this output in a file called Channel.

create a receiver Program.

1. Receiver Program should read the input from channel file.
2. Apply hamming code on the binary data to check for errors.
3. If there is an error display the Position of error
4. Else remove the redundant bits and convert the binary data to ascii and display output.

Sender:

```
String = input("Enter String:")
S = ''.join(format(ord(i), '08b') for i in String)
rb = 0
for i in range(len(s))
        if (2**rb >= len(s) + i + 1)
            rb = i
            break
    m = len(s) + rb
    i = []
    Pos = []
    c = 0
    s = s[::-1]
for i in range(rb)
        Pos.append(2**i)
    for i in range(m):
```

```
        if (i+1) in Pos:
            l.insert(i, 'p')
        else
            l.insert(i, int(s[c]))
            c+=1
Print("Parity bit Positions :", Pos)
Print("Initial encoded data with 'p' at
Parity Positions:", l)
for P in Pos:
    count=0
    i=P-1
    while i<m:
        count+=l[i:i+P].count(1)
        i+=2*P
    l[P-1]=1 if count%2!=0 else 0
Print("Final encoded data with Parity
bits:", l[::-1])
f=open("code.txt","w")
f.close()
f=open("original.txt","w")
f.write(str(l[::-1]))
f.close
```

receiver.py

```
f=open("code.txt","r")
x=f.readline()
Print("E:code",x)
e=open("original.txt","r")
y=e.readline()
Print("D locate:",y)
l=x.strip("[").string("]").strip("'\n").split(",")
for i in l:
    i.strip("'")
c=[]
Pos=[], rb=0
for i in range(len(l)):
    if(2**i>=len(r)+i+1)
        rb=i
        break
for i in range(rb)
    Pos.append(2**i)
Print("POSITIONS:", Pos)
q=[]
for i in l[::-1]:
    if(i=="1" or i=="1"):
        q.append(1)
    else:
        q.append(0)
for P to Pos:
    count=0
    i=P-1
    where i<len(l):
        count=q[i:i+P].count(1)
        i+=2*P
    c.append(0) if count%2==0 else
c.append(1)
```

```
else
    q.append(1)
for P in Pos:
    count = 0
    i = P-1
    while i < len(q):
        count = q[i:i+P].count(1)
        i += 2*P
    c.append(0) if count%2 == 0 else
c.append(1)
Print("Binary", c[1:-1])
change = 0
w = q[::-1]
for i in range(len(c[::-1])):
        change += c[i]*2**i
q[change-1] = 0 if q[change-1] == 1
else :
    Print("Error", change-1) if change-1 == 1 else
    Print("corrected", q)
```

Result:

That the implemented and executed successfully.

27/6/24